Section 11 - Functions

Information

- Title: `Section 11 - Functions`
- Authors: `ETIENNE KOA`

Les fonctions

- Fonction
  - définition
  - prototype
  - Paramètres et passage par valeur
  - déclaration `return`
  - valeurs des paramètres par défaut
  - surcharge
  - passer des tableaux à la fonction
  - passage par référence
  - Fonctions `inline`
  - Type de retour « auto »
  - fonctions récursives

```
int main() {

 // read input
   statement1;
   statement2;
   statement3;
   statement4;

 // process input
   statement5;
   statement6;
   statement7;

 // provide output
   statement8;
   statement9;
   statement10;

   return 0;
}
```

Modularized Code

```
int main() {

 // read input
   read_input();

 // process input
   process_input();

 // provide output
   provide_output();

   return 0;
}
```

Figure 1: function

Qu'est-ce qu'une fonction ?

- Programmes `C++`
  - Bibliothèques standards `C++` (fonctions et classes)
  - Bibliothèques tierces (fonctions et classes)
  - Nos propres fonctions et classes

```
int main() {

   read_input();

   process_input();

   provide_output();

   return 0;
}
```

```
read_input() {
   statement1;
   statement2;
   statement3;
   statement4;
}

process_input() {
   statement5;
   statement6;
   statement7;
}

provide_output() {
   statement8;
   statement9;
   statement10;
}
```

Figure 2: function

- Des fonctions permettent la modularisation d'un programme
  - Séparez le code en unités logiques autonomes
  - Ces unités peuvent être réutilisées
- Analogie huées/travailleurs
  - Écrivez votre code la spécification de la fonction
  - Comprendre ce que fait la fonction
  - Comprendre de quelles informations la fonction a besoin
  - Comprendre ce que renvoie la fonction
  - Comprendre les contraintes de performances
- Ne vous inquiétez pas du fonctionnement interne de la fonction
  - Sauf si c'est vous qui écrivez la fonction !
- Exemple `<cmath>`
  - Calculs mathématiques courants
  - Fonction globale appelée :
- Fonctions définies par l'utilisateur
  - Nous pouvons définir nos propres fonctions
  - Voici un aperçu
- Fonctions définies par l'utilisateur
  - Renvoie zéro si l'un des arguments est négatif

Mathematics functions

```
function_name(argument);
function_name(argument1, argument2, …);

cout << sqrt(400.0) << endl;      // 20.0
double result;
result = pow(2.0, 3.0);           // 2.0^3.0
```

Figure 3: function

```
/* This is a function that expects two integers a and b
   It calculates the sum of a and b and returns it to the caller
   Note that we specify that the function returns an int
*/

int add_numbers(int a, int b)
{
    return a + b;
}

// I can call the function and use the value that is returns

cout << add_numbers(20, 40);
```

Figure 4: function

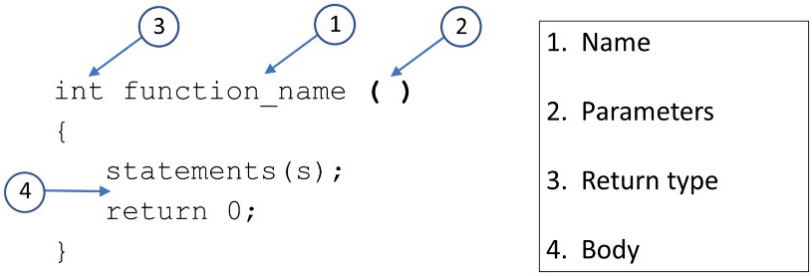
```
/* This is a function that expects two integers a and b
   It calculates the sum of a and b and returns it to the caller
   Only if a or b are non-negative. Otherwise, it returns 0
   Note that we specify that the function returns an int
*/

int add_numbers(int a, int b)
{
    if (a < 0 || b < 0)
        return 0;
    else
        return a + b;
}
```

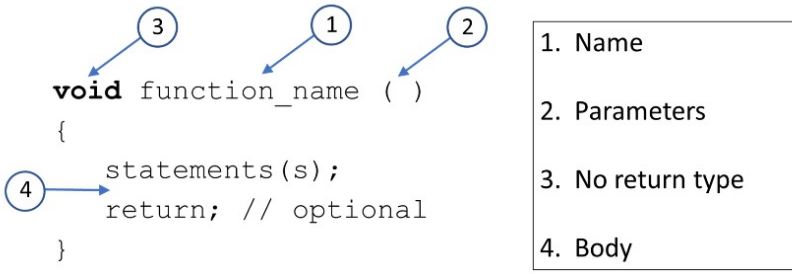
Figure 5: function

Définition de la fonction

- nom
  - le nom de la fonction
  - mêmes règles que pour les variables
  - devrait être significatif
  - généralement un verbe ou une phrase verbale
- liste des paramètres
  - La variable passée dans la fonction
  - leurs types doivent être précisés
- type de retour
  - le type de données renvoyées par la fonction
- corps
  - les instructions exécutées lorsque la fonction est appelée.
  - entre accolades `{}`



- Exemple sans paramètres



- Exemple sans type de retour (void)

```
void function_name(int a, std::string b)
{
   statements(s);
   return; // optional
}
```

- Exemple avec plusieurs paramètres

```
void say_hello () {
   cout << "Hello" << en
}
```

- Une fonction sans type de retour et sans paramètres

Appeler une fonction

```
void say_hello () {
   cout << "Hello" << endl;
}

int main() {
   for (int i{1} i<=10; ++i)
      say_hello();
   return 0;
}
```

```
void say_world () {
   cout << " World" << endl;
   cout << " Bye from say_world" << endl;
}

void say_hello () {
   cout << "Hello" << endl;
   say_world();
   cout << " Bye from say_hello" << endl;
}

int main() {
   say_hello();
   cout << " Bye from main" << endl;
   return 0;
}
```

```
Hello
World
Bye from say_world
Bye from say_hello
Bye from main
```

- Les fonctions peuvent appeler d'autres fonctions
- Le compilateur doit connaître les détails de la fonction « AVANT » qu'elle soit appelée !

```
int main() {
   say_hello();  // called BEFORE it is defined ERROR
   return 0;
}

void say_hello ()
{
   cout << "Hello" << endl;
}
```

Figure 6: function

Prototypes de fonctions

- **Le compilateur doit « connaître » une fonction avant de l'utiliser**
- Définir les fonctions avant de les appeler
  - OK pour les petits programmes
  - Pas une solution pratique pour les programmes plus importants
- Utiliser des prototypes de fonctions
  - Indique au compilateur ce qu'il doit savoir sans une définition complète de la fonction
  - Également appelées déclarations anticipées
  - Placé en début de programme
  - Également utilisé dans nos propres fichiers d'en-tête `(.h)` - nous en reparlerons plus tard
- Exemple

```
int function_name();   // prototype


int function_name()
{
   statements(s);
   return 0;
}
```

```
int function_name(int);    // prototype
                           // or
int function_name(int a);  // prototype
```

```
int function_name(int a) {
   statements(s);
   return 0;
}
```

```
void function_name(); // prototype
```

```
void function_name()
{
   statements(s);
   return; // optional
}
```

```
void function_name(int a, std::string b);
// or
void function_name(int, std::string);
```

```
void function_name(int a, std::string b)
{
   statements(s);
   return; // optional
}
```

```
void say_hello();

void say_hello() {
    cout << "Hello" << en
}
```

- Une fonction sans type de retour et sans paramètres

```
void say_hello();

int main() {
    say_hello();            // OK
    say_hello(100);        // Error
    cout << say_hello();  // Error
                          // No return value
    return 0;
}
```

- Appel d'une fonction

```
void say_hello(); // prototype
void say_world(); // prototype

int main() {
    say_hello();
    cout << " Bye from main" << endl;
    return 0;
}
```

```
void say_world () {
    cout << " World" << endl;
    cout << " Bye from say_world" << endl;
}

void say_hello () {
    cout << "Hello" << endl;
    say_world();
    cout << " Bye from say_hello" << endl;
}
```

- Exemple

Paramètres des fonctions

- Lorsque nous appelons une fonction, nous pouvons transmettre des données à cette fonction
- Dans l'appel de fonction, ils sont appelés arguments
- Dans la définition de la fonction, ils sont appelés paramètres.
- Ils doivent correspondre en nombre, en ordre et en type.

```
int add_numbers(int, int);        // prototype

int main() {
   int result {0};
   result = add_numbers(100,200);  // call
   return 0;
}

int add_numbers(int a, int b) {  // definition
   return a + b;
}
```

- Exemple

```
void say_hello(std::string name) {
   cout << "Hello " << name << endl;
}

say_hello("Frank");

std::string my_dog {"Buster"};
say_hello(my_dog);
```

- Passage par valeur
  - Lorsque vous transmettez des données dans une fonction, elles sont transmises par valeur.
  - Une copie des données est transmise à la fonction.
  - Quelles que soient les modifications que vous apportez au paramètre dans la fonction, cela n'affecte PAS l'argument qui a été transmis.
- Paramètres formels et réels
  - `Paramètres formels` - les paramètres définis dans l'en-tête de la fonction.
  - `Paramètres réels` - le paramètre utilisé dans l'appel de fonction, les arguments.
- Example
- Déclaration de retour de fonction
  - si une fonction renvoie une valeur alors elle doit utiliser une instruction `return` qui renvoie une valeur.
  - Si une fonction ne renvoie pas de valeur `(void)` alors l'instruction `return` est facultative.
  - L'instruction `return` peut apparaître n'importe où dans le corps de la fonction.
  - L'instruction `return` quitte immédiatement la fonction.
  - Nous pouvons avoir plusieurs instructions `return` dans une fonction

```
void param_test(int formal) { // formal is a copy of actual
   cout << formal << endl;     / 50
   formal = 100;              // only changes the local copy
   cout << formal << endl;    // 100
}

int main() {
   int actual {50};
   cout << actual << endl;    // 50
   param_test(actual);        // pass in 50 to param_test
   cout << actual << endl;    // 50   - did not change
   return 0
}
```

Figure 7: function

    * Évitez de nombreuses instructions de retour dans une fonction.
  - La valeur de retour est le résultat de l'appel de fonction.

Valeurs des arguments par défaut

- Lorsqu'une fonction est appelée, tous les arguments doivent être fournis.
- Parfois, certains arguments ont la plupart du temps les mêmes valeurs.
- Nous pouvons dire au compilateur d'utiliser les valeurs par défaut si les arguments ne sont pas fournis.
- Les valeurs par défaut peuvent être dans le prototype ou la définition, pas les deux
  - meilleure pratique - dans le prototype.
  - doit apparaître à la fin de la liste des paramètres.
- Peut avoir plusieurs valeurs par défaut
  - doit apparaître consécutivement à la fin de la liste des paramètres.
- Exemple - aucun argument par défaut

```
double calc_cost(double base_cost, double tax_rate);

double calc_cost(double base_cost, double tax_rate) {
   return base_cost += (base_cost * tax_rate);
}

int main() {
   double cost {0};
   cost = calc_cost(100.0, 0.06);
   return 0;
}
```

```
double calc_cost(double base_cost, double tax_rate = 0.06);

double calc_cost(double base_cost, double tax_rate) {
   return base_cost += (base_cost * tax_rate);
}

int main() {
   double cost {0};
   cost = calc_cost(200.0);        // will use the default tax
   cost = calc_cost (100.0, 0.08);      // will use 0.08 not the defauly
   return 0;
}
```

- Exemple - argument par défaut unique

```
double calc_cost(double base_cost, double tax_rate = 0.06, double shipping = 3

double calc_cost(double base_cost, double tax_rate, double shipping) {
   return base_cost += (base_cost * tax_rate) + shipping;
}

int main() {
   double cost {0};
   cost = calc_cost (100.0, 0.08, 4.25);  // will use no defaults
   cost = calc_cost(100.0, 0.08);         // will use default shipping
   cost = calc_cost(200.0);          // will use default tax and shipping
return 0;
}
```

- Exemple - plusieurs arguments par défaut

Fonctions de surcharge

- Nous pouvons avoir des fonctions qui ont une liste de paramètres différente qui portent le même nom.
- Mécanisme d'abstraction puisqu'on peut simplement penser `imprimer` par exemple.
- Un type de « polymorphisme »
  - Nous pouvons faire fonctionner le même nom avec différents types de données pour exécuter le comportement.
- Le compilateur doit être capable de distinguer les fonctions en fonction des listes de paramètres et des arguments fournis.

```
int add_numbers(int, int);        // add ints
double add_numbers(double, double);  // add doubles

int main() {
   cout << add_numbers(10,20) << endl;    // integer
   cout << add_numbers(10.0, 20.0) << endl; // double
 return 0;
}
```

- Exemple

```
int add_numbers(int a, int b) {
   return a + b;
}

double add_numbers(double a, double b) {
   return a + b;
}
```

```
void display(int n);
void display(double d);
void display(std::string s);
void display(std::string s, std::string t);
void display(std::vector<int> v);
void display(std::vector<std::string> v);
```

```
int    get_value();
double get_value();

// Error

cout << get_value() << endl; // which one?
```

- Le type de retour n'est pas pris en compte

Passer des tableaux aux fonctions

- Nous pouvons passer un tableau à une fonction en fournissant des crochets dans la description formelle du paramètre.
- Les éléments du tableau ne sont « PAS » copiés.
- Puisque le nom du tableau correspond à l'emplacement du tableau en mémoire, cette adresse est ce qui est copié.
- La fonction n'a donc aucune idée du nombre d'éléments - cette adresse est ce qui est copié.
- La fonction n'a donc aucune idée du nombre d'éléments dans le tableau puisque tout ce qu'elle sait, c'est l'emplacement du premier élément (le nom du tableau).

```
void print_array(int numbers []);

int main() {
   int my_numbers[] {1,2,3,4,5};
   print_array(my_numbers);
   return 0;
}

void print_array(int numbers []) {
   // Doesn't know how many elements are in the array???
   // we need to pass in the size!!
}
```

- Exemple

```
void print_array(int numbers [], size_t size);

int main() {
   int my_numbers[] {1,2,3,4,5};
   print_array(my_numbers, 5);   / 1 2 3 4 5
   return 0;
}

void print_array(int numbers [], size_t size) {
   for (size_t i{0}; i < size; ++i )
      cout << numbers[i] << endl;
}
```

- Puisque nous transmettons l'emplacement du tableau.

```
void zero_array(int numbers [], size_t size) {
   for (size_t i{0}; i < size; ++i )
      numbers[i] = 0;                  // zero out array eleme
}
int main() {
   int my_numbers[] {1,2,3,4,5};
   zero_array(my_numbers, 5);       // my_numbers is now zero
   print_array(my_numbers, 5);     // 0 0 0 0 0
   return 0;
}
```

  - La fonction peut modifier le tableau réel !

- paramètres const
  - Nous pouvons dire au compilateur que les paramètres de la fonction sont `const` (lecture seule).
  - Cela pourrait être utile pour la fonction `print-array` car elle ne devrait `PAS` modifier le tableau.

```
void print_array(const int numbers [], size_t size) {
   for (size_t i{0}; i < size; ++i )
      cout << numbers[i] << endl;
   numbers[i] = 0;   // any attempt to modify the array
                     // will result in a compiler error
}
```

Passer par référence

- Parfois, nous voulons pouvoir modifier le paramètre réel depuis le corps de la fonction.
- Pour y parvenir, nous avons besoin de l'emplacement ou de l'adresse du paramètre réel.
- Nous avons vu quel est l'effet avec le tableau, mais qu'en est-il des autres types de variables ?
- Nous pouvons utiliser des paramètres de référence pour indiquer au compilateur de transmettre une référence au paramètre réel.
- Le paramètre formel sera désormais un alias pour le paramètre réel.

```
void scale_number(int &num);      // prototype

int main() {
   int number {1000};
   scale_number(number);         // call
   cout << number << endl;       // 100
   return 0;
}

void scale_number(int &num) {   // definition
   if (num > 100)
      num = 100;
}
```

- Exemple

```
void swap(int &a, int &b);

int main() {
   int x{10}, y{20};
   cout << x << " " << y << endl;   // 10 20
   swap(x, y);
   cout << x << " " << y << endl;   // 20 10
   return 0;
}

void swap(int &a, int &b) {
   int temp = a;
   a = b;
   b = temp;
}
```

```
void print(std::vector<int> v);

int main() {
   std::vector<int> data {1,2,3,4,5};
   print(data);                    // 1 2 3 4 5
   return 0;
}

void print(std::vector<int> v) {
   for (auto num: v)
      cout << num << endl;
}
```

- Exemple vectoriel - passage par valeur

```
void print(std::vector<int> &v);

int main() {
   std::vector<int> data {1,2,3,4,5};
   print(data);                    // 1 2 3 4 5
   return 0;
}

void print(std::vector<int> &v) {
   for (auto num: v)
      cout << num << endl;
}
```

- Exemple vectoriel - passer par référence

```
void print(const std::vector<int> &v);

int main() {
   std::vector<int> data {1,2,3,4,5};
   print(data);                    // 1 2 3 4 5
   return 0;
}

void print(const std::vector<int> &v) {
   v.at(0) = 200;                          // ERRO
   for (auto num: v)
      cout << num << endl;
}
```

- Exemple vectoriel - passer par référence const

Règles de portée

Portée locale ou de bloc

- Identifiants déclarés dans le bloc {}
- Uniquement visible dans le bloc {} où il a été déclaré.
- Les variables locales de la fonction ne sont actives que pendant l'exécution de la fonction.

- Les variables locales ne sont PAS conservées entre les appels de fonction.
- Avec les blocs imbriqués, les blocs intérieurs peuvent « voir » mais les blocs extérieurs « voient » à l'intérieur.

Variables locales statiques

- Déclaré avec les qualificatifs « statiques »

```
static int value {10};
```

- La valeur EST conservée entre les appels de fonction.
- Initialisé uniquement lors du premier appel de la fonction

Portée Global

- Identifiant déclaré en dehors de toute fonction ou classe.
- Visible par toutes les parties du programme après la déclaration de l'identifiant global.
- Les constantes globales sont OK.
- Meilleure pratique - n'utilisez pas de variables globales.

Comment fonctionnent les appels de fonction ?

La fonction utilise la 'pile d'appels de fonction' + Analogue à une pile de livres + LIFO - Dernier entré, premier sorti + push et pop

Cadre de pile ou enregistrement d'activation + Les fonctions doivent rendre le contrôle à la fonction qui l'a appelé. + Chaque fois qu'une fonction est appelée, nous créons un nouvel enregistrement d'activation et le plaçons sur la pile. + Lorsqu'une fonction se termine, nous ouvrons l'enregistrement d'activation et revenons. + Les variables locales et les paramètres de fonction sont alloués sur la pile.

La taille de la pile est finie - Stack Overflow

Fonctions en ligne

- Les appels de fonction entraînent une certaine surcharge.
- Vous avez vu ce qui se passe sur la pile d'appels
- Parfois nous avons des fonctions simples
- On peut suggérer au compilateur de les compiler 'inline'
  - éviter la surcharge des appels de fonction
  - générer du code assembleur en ligne
  - plus rapide
  - pourrait provoquer une surcharge du code
- Les optimisations des compilateurs sont très sophistiquées
  - Sera probablement intégré même sans votre suggestion

Exemple

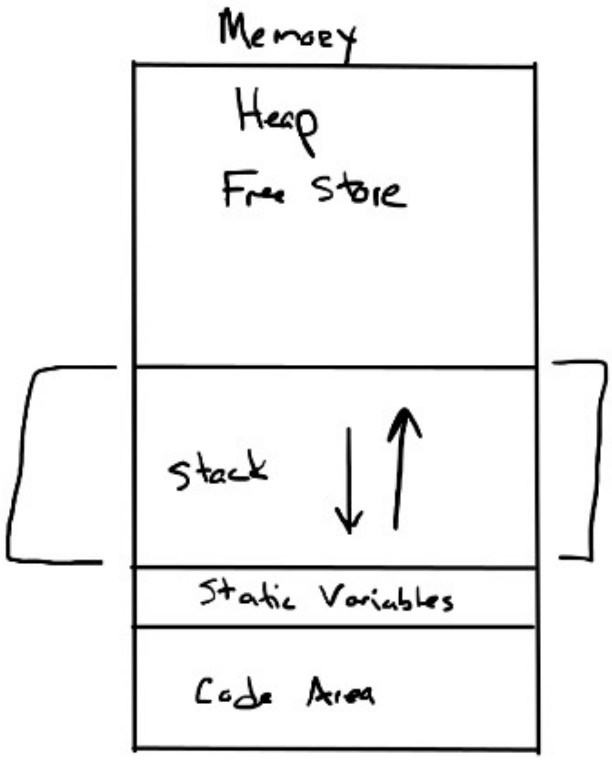


Figure 8: function

```
inline int add_numbers(int a, int b) {// definition
   return a + b;
}

int main() {
   int result {0};
   result = add_numbers(100,200); // call
   return 0;
}
```

Figure 9: function

Fonctions récursives

- Une fonction récursive est une fonction qui s'appelle elle-même.
  - Soit directement, soit indirectement via une autre fonction
- Résolution de problèmes récursive
  - Cas de base
  - Divisez le reste du problème en sous-problème et effectuez un appel récursif
- Il existe de nombreux problèmes qui se prêtent à des solutions récursives
- Mathématiques - Factorielle, Fibonacci, Fractales...

```
0! = 1
n! = n * (n-1)!
```

- **Base case:**
  - factorial(0) = 1
- **Recursive case:**
  - factorial(n) = n * factorial(n-1)

Exemple - Factorielle

```
unsigned long long factorial(unsigned long long n) {
   if (n == 0)
      return 1;                    // base case
   return n * factorial(n-1);      // recursive case
}

int main() {
   cout << factorial(8) << endl;  // 40320
   return 0;
}
```

```
Fib(0) = 0
Fib(1) = 1
Fib(n) = Fib(n-1) + Fib(n-2)
```

- **Base case:**
  - Fib(0) = 0
  - Fib(1) = 1
- **Recursive case:**
  - Fib(n) = Fib(n-1) + Fib(n-2)

Exemple - Fibonacci

```
unsigned long long fibonacci(unsigned long long n) {
   if (n <= 1)
      return n;              // base cases
   return fibonacci(n-1) + fibonacci(n-2); // recursion
}

int main() {
   cout << fibonacci(30) << endl; // 832040
   return 0;
}
```

Exemple - Factorielle

Notes IMPORTANTES

- Si la récursivité ne s'arrête pas, vous aurez une récursivité infinie.
- La récursivité peut nécessiter beaucoup de ressources.
- N'oubliez pas le(s) cas de base
  - Il termine la récursion
- N'utilisez des solutions récursives que lorsque cela a du sens
- Tout ce qui peut être fait de manière récursive peut être fait de manière itérative
  - Erreur de débordement de pile